INF333 2025-2026 Spring Semester

Fanatics
Önder Akyüz <23401064@ogr.gsu.edu.tr>
Gökay Kayra Bereket <23401065@ogr.gsu.edu.tr>

Homework I
Design Document

Please provide answers inline in a `quote` environment.

1 Preliminaries

Q1: If you have any preliminary comments on your submission, notes for the TAs, or extra credit, please give them here.

> We worked together on this homework. Önder did the alarm clock and advanced scheduler parts, Kayra did priority scheduling and donation. All 27 tests pass.

Q2: Please cite any offline or online sources you consulted while preparing your submission, other than the Pintos documentation, course text, and lecture notes.

> Outside of the course materials, I consulted the following resources to better understand the synchronization primitives and the MLFQS fixed-point arithmetic:
>
> - **Stack Overflow:** Used for clarifying general C syntax issues and understanding how `list_entry` macros work in C.
> - **YouTube (Operating Systems Tutorials):** Watched various conceptual videos on Priority Inversion and Priority Donation to visualize the lock-waiting chains.
> - **GNU C Library (glibc) Documentation:** Consulted for standard integer types and bitwise operations used in fixed-point calculations.

2 Sleep

2.1 Data Structures

Q3: Copy here the declaration of each new or changed 'struct' or 'struct' member, global or static variable, 'typedef', or enumeration.

Identify the purpose of each in 25 words or less.

In `thread.h` inside `struct thread`:
`int64_t wakeup_tick;` / Tick when the thread should wake up, used for the alarm clock implementation.
`int base_priority;` / Stores the original priority to restore it after all priority donations are released.
`struct lock *wait_lock;` / The lock the thread is currently waiting for; used to handle nested priority donations.
`struct list donation;` / List of threads currently donating their priority to this thread.
`struct list_elem donation_element;` / List element used to track this thread in a lock holder's donation list.
`int nice;` / Determines how "nice" a thread is to others, affecting its priority in MLFQS.
`int recent_cpu;` / Estimates how much CPU time the thread has recently received in fixed-point format.

In `timer.c` part:
`static struct list thrsleep_list;` / Global list that stores all currently sleeping threads, ordered by their wake-up ticks.
`wakeup_comparison();` / Comparison function used to keep the `thrsleep_list` sorted by `wakeup_tick` for efficient insertion.

In `thread.c` as a global/static variable:
`static int load_avg;` / Estimates the average number of threads ready to run over the past minute in fixed-point.

2.2 Algorithms

Q4: Briefly describe your implementation of `thread_join()` and how it interacts with thread termination.

thread_join() is not part of this assignment.

Q5: What steps are taken to minimize the amount of time spent in the timer interrupt

handler?

The sleep list is sorted by wakeup time, so we only need to look at the front. Once we see a thread that shouldn't wake up we could stop checking. We don't need to go through the whole list.

2.3 Synchronization

Q6: Consider parent thread `P` with child thread `C`. How do you ensure proper synchronization and avoid race conditions when `P` calls `wait(C)` before `C` exits? After `C` exits? How do you ensure that all resources are freed in each case? How about when `P` terminates without waiting, before `C` exits? After `C` exits? Are there any special cases?

We didn't implement thread_join.

Q7: How are race conditions avoided when multiple threads call `timer_sleep()` simultaneously?

We turn off interrupts before modifying thrsleep_list and calling thread_block(). Like this, only one thread can modify the list at a same time.

Q8: How are race conditions avoided when a timer interrupt occurs during a call to `timer_sleep()`?

We disable interrupts at the start of timer_sleep() before inserting into the list and blocking. In this way, a timer interrupt can't be between the list insertion and thread_block(). Otherwise things would break.

2.4 Rationale

Q9: Critique your design, pointing out advantages and disadvantages in your design choices.

Advantage: The sorted list makes the interrupt handler fast because we only check the front.

Disadvantage: Inserting into a sorted list takes O(n) time where n is the number of sleeping threads. However, it is not a big problem.

3 Priority Scheduling

3.1 Data Structures

Q10: Copy here the declaration of each new or changed 'struct' or 'struct' member, global or static variable, 'typedef', or enumeration. Identify the purpose of each in 25 words or less

In thread.h inside struct thread:
int base_priority; / Stores the thread's original priority, allowing it to revert back once donated priorities are no longer needed.
struct lock *wait_lock; / Points to the lock this thread is currently waiting for, which is essential for following the donation chain.
struct list donation; / A list containing all the threads that have donated their higher priority to this specific thread.
struct list_elem donation_element; / The list element used to insert this thread into the donation list of the thread holding the requested lock.

Q11: Describe the sequence of events when a call to `lock_acquire()` causes a priority donation. How is nested donation handled?

When a thread tries to acquire a held lock, the following happens:

1. It sets its `wait_lock` to remember what it's blocked on.
2. It inserts itself into the lock holder's `donation` list.
3. If its priority is higher, it donates this priority to the holder so the lock can be released faster.
4. Finally, the thread goes to sleep via `sema_down()`.

Nested Donation Handling:
To handle chains (like A waits for B, B waits for C), I use a `while` loop. When a priority is donated, I check if the holder is also waiting for another lock. By following the `wait_lock->holder` pointers, I pass the priority up the chain. I set a maximum depth limit of 8 iterations to prevent infinite loops and keep the system fast.

Q12: Describe the sequence of events when `lock_release()` is called on a lock that a higher-priority thread is waiting for.

When a thread calls `lock_release()`, it no longer needs the priority donated for that lock. The steps are:

1. I loop through the `donation` list and remove all threads waiting for this specific lock.

2. I recalculate the thread's priority. If it still holds other locks with donors, its priority becomes the highest remaining donation. Otherwise, it drops back to its `base_priority`.

3. I clear the lock's holder and call `sema_up()`. Since the semaphore's wait list is sorted, this automatically wakes up the highest-priority waiter.

4. Since the current thread's priority just dropped and the awakened thread has a higher priority, the current thread immediately yields the CPU to let it run.

3.2 Synchronization

Q13: Describe a potential race in `thread_set_priority()` and explain how your implementation avoids it. Can you use a lock to avoid this race?

A race condition can happen if a timer interrupt fires right in the middle of `thread_set_priority()`. The scheduler might read a half-updated priority and make a wrong decision before the thread finishes recalculating its donations.

To avoid this, I disabled the interrupts using `intr_disable()` before updating the priority and restored them right after. This makes the update atomic.

We CANNOT use a lock here. Locks can put the caller to sleep if they are busy. The timer interrupt handler also reads thread priorities, and since interrupt handlers are not allowed to sleep, waiting for a lock would crash the kernel.

3.3 Rationale

Q14: Why did you choose this design? In what ways is it superior to another design you considered?

I chose to keep a `donation` list inside the thread structure to track all the threads donating to it.

Another design I considered was simply storing the highest donated priority in a single integer variable instead of a list. However, I quickly realized that this simple alternative completely fails when a thread holds multiple locks. If a thread releases one lock, it needs to know the *second* highest donated

priority from the other locks to fall back to. Without a list, that history is lost.

My list-based design is superior because it keeps a complete record of who donated what. When `lock_release()` is called, I can just remove the threads waiting for that specific lock from the list and easily recalculate the new maximum priority from the remaining donors. Also, putting a hard depth limit of 8 for nested donations keeps the system fast and safe from infinite loops.

4 Advanced Scheduler

4.1 Data Structures

Q15: Copy here the declaration of each new or changed 'struct' or 'struct' member, global or static variable, 'typedef', or enumeration. Identify the purpose of each in 25 words or less.

In thread.h part: int nice; / niceness between -20 and 20, int recent_cpu; / cpu time used recently as fixed-point

In thread.c part: static int load_avg; / system load average as fixed-point

fixed-point.h new file: There are some functions for converting between int and fixed-point and doing arithmetic on fixed-point numbers.

4.2 Algorithms

Q16: Suppose threads A, B, and C have nice values 0, 1, and 2. Each has a recent_cpu value of 0. Fill in the table below showing the scheduling decision and the priority and recent_cpu values for each thread after each given number of timer ticks:

timer	recent_cpu			priority			thread
ticks	A	B	C	A	B	C	to run
0	0	0	0	63	61	59	A
4	4	0	0	62	61	59	A
8	8	0	0	61	61	59	B
12	8	4	0	61	60	59	A
16	12	4	0	60	60	59	B
20	12	8	0	60	59	59	A
24	16	8	0	59	59	59	C
28	16	8	4	59	59	58	A
32	20	8	4	58	59	58	B
36	20	12	4	58	58	58	C

Q17: Did any ambiguities in the scheduler specification make values in the table uncertain? If so, what rule did you use to resolve them? Does this match the behavior of your scheduler?

> When threads have the same priority, it is not clear that which thread should run. We just go with round robin, so whichever has been waiting longest. Our scheduler does this because we always take from the front of the sorted ready list.

Q18: How is the way you divided the cost of scheduling between code inside and outside interrupt context likely to affect performance?

> All the mlfqs updates happen inside the timer interrupt. We increment recent_cpu every tick, update load_avg every second and recalculate priorities every 4 ticks. It is simple but may be slow with lots of threads because we work of them each time.

4.3 Rationale

Q19: Briefly critique your design, pointing out advantages and disadvantages in your design choices. If you were to have extra time to work on this part of the project, how might you choose to refine or improve your design?

> Advantage: Simple design, we just followed the formulas so it is easy to understand and debug.
>
> Disadvantage: We loop over all threads every 4 ticks and every second which is O(n). Than, it could be slow if there are many threads.
>
> With extra time, we would only update threads whose recent_cpu actually changed instead of going through all of them every time.

Q20: The assignment explains arithmetic for fixed-point math in detail, but it leaves it open to you to implement it. Why did you decide to implement it the way you did? If you created an abstraction layer for fixed-point math, that is, an abstract data type and/or a set of functions or macros to manipulate fixed-point numbers, why did you do so? If not, why not?

> We made a separate fixed-point.h file with functions for the math. Functions are easier to read than macros and the compiler inlines them anyway so it doesn't matter. We used 17.14 format because the assignment told us to use that format.

5 Survey Questions

Answering these questions is optional, but it will help us improve the course in future quarters. Feel free to tell us anything you want–these questions are just to spur your thoughts. You may also choose to respond anonymously in the course evaluations at the end of the quarter.

Q1: In your opinion, was this assignment, or any one of the three problems in it, too easy or too hard? Did it take too long or too little time?

Alarm clock was less difficult compared to the other parts.

Q2: Did you find that working on a particular part of the assignment gave you greater insight into some aspect of OS design?

Working on Priority Donation gave me the greatest insight into OS design. Before this assignment, I thought scheduling was as simple as picking the highest-priority thread from a ready list. Implementing nested donations showed me how deeply interconnected threads become when they share resources like locks.

Additionally, handling race conditions in `thread_set_priority()` was a huge learning moment. Realizing that I had to disable hardware interrupts instead of just using a lock—because the timer interrupt handler cannot sleep—really made me understand the harsh, low-level reality of kernel concurrency.

Q3: Is there some particular fact or hint we should give students in future quarters to help them solve the problems? Conversely, did you find any of our guidance to be misleading?

Starting this project from scratch was a bit overwhelming. It would help if some extra resources or guides were shared to point students in the right direction.

Q4: Do you have any suggestions for the TAs to more effectively assist students, either for future quarters or the remaining projects?

One suggestion would be to provide a small visual diagram in the early discussion sessions showing exactly how `list_elem` is embedded inside `struct thread` or `struct lock`. It takes a while to get used to the Pintos list API, and a visual guide would save future students a lot of trial and error before they even start coding.

Q5: Any other comments?

Good project, we learned a lot.